

Diabetic vs Control RNA-seq → ceRNA Network:

15-Day Project Plan

A hands-on, from-scratch bioinformatics project that mirrors the UAEU RA role: RNA-seq of **diabetic vs control** samples, miRNA/lncRNA identification, gene-target prediction, and ceRNA-network construction — run entirely locally on macOS via conda and the command line (no Galaxy).

0. The dataset

GSE164416 — Multi-omics profiling of **living human pancreatic islet donors** reveals **heterogeneous beta cell trajectories toward type 2 diabetes** (Wigger et al., 2021).

- **Organism:** *Homo sapiens* (pancreatic **islets**, **laser-capture microdissected**)
- **Platform:** **Illumina HiSeq 2500**, **bulk RNA-seq**
- **Design:** **133 donors** — **18 ND** (non-diabetic / control), **41 IGT** (impaired glucose tolerance), **35 T3cD**, **39 T2D** (type-2 diabetic)
- **Raw data:** **SRA PRJNA690574** / **SRP300812** (**FASTQ** per sample)
- **Processed data:** **GSE164416_DP_htseq_counts.txt.gz** (5.2 MB) — a **full gene-level count matrix**
- **GEO:** <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE164416>

Why this one: It's a **clean human control (ND) vs diabetic (T2D)** contrast, and the **intermediate IGT** group gives you a **third** arm that maps conceptually onto the "control / diabetic / treatment" three-group structure in the job description. It's a **polyA/total bulk library**, so it captures **mRNAs and lncRNAs** (both annotated in GENCODE), which is exactly what the **ceRNA layer needs**. Note: standard RNA-seq does **not** capture mature miRNAs (those need small-RNA-seq), so the **miRNA layer of the ceRNA network is built by computational prediction from curated databases** — this is the standard approach in the ceRNA literature and is what the plan below does.

The two-track strategy (important — read this first)

Aligning **130+** human samples on a **16 GB laptop is not realistic**. So you'll run **two tracks in parallel**, and this **is a strength, not a shortcut** — it's exactly how you'd honestly describe it in an interview:

- **Track A — Pipeline demonstration (proves you can run the tools):** Download a small **subset of ~6 ND + 6 T2D** FASTQ samples and run the **full** raw pipeline on them: QC → trim → align → count. This produces your own count matrix and demonstrates command-line competence end to end.
- **Track B — Biology (proves you can find real signal):** Use the **full published HTSeq count matrix** for the DESeq2 differential-expression, lncRNA/mRNA, and ceRNA-network analysis, so your results are statistically solid (**12 samples is underpowered for DE**).

You then show in the README that your Track-A counts on shared samples correlate tightly with the published counts — this validates your pipeline. Best of both worlds.

Repository layout (build this on Day 1)

```
diabetes-cerna-rnaseq/
├── README.md
├── environment.yml           # conda spec for the whole project
├── LICENSE
├── .gitignore               # ignore data/, results/large files, *.bam, *.fastq.gz
├── data/
│   ├── raw/                # FASTQ (gitignored)
│   ├── reference/          # genome index + GTF (gitignored)
│   └── processed/           # count matrices, metadata (small → tracked)
├── scripts/
│   ├── 01_download.sh
│   ├── 02_qc_trim.sh
│   ├── 03_align_count.sh
│   ├── 04_deseq2.R
│   ├── 05_biotype_split.R
│   ├── 06_enrichment.R
│   ├── 07_cerna_predict.R
│   └── 08_network.R
├── results/
│   ├── qc/                 # MultiQC reports
│   ├── tables/             # DE tables, ceRNA edges
│   └── figures/            # volcano, heatmap, network
└── docs/
    └── methods.md
```

.gitignore essentials (never commit big or raw files):

```
data/raw/
data/reference/
*.bam
*.sam
*.fastq
*.fastq.gz
results/**/*.*pdf
.Rhistory
__pycache__/
```

Day-by-day plan

Each day lists **Goal** → **Tools** → **Commands** → **Expected output**. Steps that can exceed ~4 hours are marked **>4h** with a mitigation.

Day 1 — Environment, repo, and Git/GitHub setup

Goal: A working conda toolchain and an initialized GitHub repo.

Tools: Miniforge (conda), git, GitHub.

Commands

Install Miniforge (the community conda distribution that plays best with bioconda on macOS). On Apple Silicon this matters — many bioinformatics tools are only built for Intel (`osx-64`), so we force that subdir and run under Rosetta:

```
# Apple Silicon: install Rosetta once (skip on Intel Macs)
softwareupdate --install-rosetta --agree-to-license

# Install Miniforge
curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-$(uname)=$(uname -m).sh"
bash Miniforge3-$(uname)=$(uname -m).sh -b
~/miniforge3/bin/conda init "$(basename "$SHELL")"
# restart your terminal after this
```

Create the project env. On Apple Silicon prefix creation with `CONDA_SUBDIR=osx-64` so bioconda packages resolve:

```
mkdir -p ~/projects/diabetes-erna-rnaseq && cd ~/projects/diabetes-erna-rnaseq

# Apple Silicon users: export CONDA_SUBDIR=osx-64 (Intel users: skip)
export CONDA_SUBDIR=osx-64

conda create -y -n rnaseq -c conda-forge -c bioconda \
  sra-tools fastqc multiqc fastp hisat2 subread samtools pysnpdb
conda activate rnaseq
conda config --env --set subdir osx-64 # pin the env so future installs match
```

Create a **separate R env** (mixing R + heavy CLI tools in one env often breaks solves):

```
conda create -y -n rstats -c conda-forge -c bioconda \
  r-base r-essentials bioconductor-deseq2 bioconductor-clusterprofiler \
  bioconductor-org.hs.eg.db bioconductor-enhancedvolcano bioconductor-multimir \
  r-pheatmap r-ggplot2 r-igraph r-dplyr r-data.table bioconductor-biomart
```

Scaffold the repo and push:

```
mkdir -p data/{raw,reference,processed} scripts results/{qc,tables,figures} docs
printf "data/raw/\ndata/reference/\n*.bam\n*.sam\n*.fastq\n.Rhistory\n" > .gitignore
git init && git add . && git commit -m "Initial project scaffold"
# create an empty repo named diabetes-erna-rnaseq on github.com, then:
git branch -M main
git remote add origin https://github.com/<your-username>/diabetes-erna-rnaseq.git
git push -u origin main
```

Export the env spec so the repo is reproducible:

```
conda env export -n rnaseq --no-builds > environment.yml
git add environment.yml && git commit -m "Add environment spec" && git push
```

Expected output: `conda activate rnaseq` works; `fastqc --version`, `hisat2 --version`, `featureCounts -v` all print versions; your empty scaffold is live on GitHub.

Day 2 — Select the subset and download FASTQ + reference

Goal: ~6 ND + 6 T2D FASTQ files locally, plus the human reference for alignment.

Tools: SRA Run Selector, pysradb, sra-tools (or ENA direct download).

Pick your samples. Open the SRA Run Selector for the study and download the metadata table: <https://www.ncbi.nlm.nih.gov/Traces/study/?acc=PRJNA690574> → "Metadata" → SraRunTable.csv. Filter to ND and T2D in the sample-title column and choose 6 of each. Example ND GSMs from GEO: GSM5009234, GSM5009235, GSM5009236, GSM5009239, GSM5009275, GSM5009280; example T2D: GSM5009230, GSM5009243, GSM5009250, GSM5009255, GSM5009257, GSM5009267. Map GSM → SRR with pysradb:

```
pysradb gsm-to-srr GSM5009234 GSM5009235 GSM5009236 GSM5009239 GSM5009275 GSM5009280 \
GSM5009230 GSM5009243 GSM5009250 GSM5009255 GSM5009257 GSM5009267 \
--saveto data/processed/subset_srr.tsv
cat data/processed/subset_srr.tsv      # note the SRR accessions
```

Download the FASTQs. 🚩 >4h >4h possible (depends on your internet — each human RNA-seq sample is ~1–4 GB; 12 samples can be 20–40 GB and several hours). Mitigations: run overnight; or drop to 4+4 samples; or pull directly from ENA (often faster than SRA):

```
# scripts/01_download.sh
mkdir -p data/raw && cd data/raw
for SRR in $(cut -f2 ../processed/subset_srr.tsv | tail -n +2); do
  prefetch "$SRR"
  fasterq-dump "$SRR" --split-files --threads 4 --outdir .
  gzip "${SRR}"_*.fastq
done
```

Download the reference (GRCh38) and annotation. Use a prebuilt HISAT2 index to avoid a multi-hour index build:

```
cd data/reference
# Prebuilt HISAT2 GRCh38 genome index (~4 GB download)
curl -L -O https://genome-idx.s3.amazonaws.com/hisat/grch38_genome.tar.gz
tar -xzf grch38_genome.tar.gz
# GENCODE annotation (GTF) — needed for counting and for lncRNA/mRNA biotypes
curl -L -O https://ftp.ebi.ac.uk/pub/databases/genCODE/GenCODE_human/release_44/
gencode.v44.annotation.gtf.gz
gunzip gencode.v44.annotation.gtf.gz
```

Also grab the published count matrix for Track B:

```
cd ../processed
curl -L -o GSE164416_counts.txt.gz \
  "https://www.ncbi.nlm.nih.gov/geo/download/?
  acc=GSE164416&format=file&file=GSE164416%5FDP%5Fhtseq%5Fcounts%2Etxt%2Egz"
gunzip GSE164416_counts.txt.gz
```

Expected output: data/raw/ holds paired FASTQ (SRRxxxx_1.fastq.gz , _2.fastq.gz) for 12 samples; data/reference/ holds the HISAT2 index (grch38/genome.*.ht2) and gencode.v44.annotation.gtf; the published count matrix is in data/processed/.

Day 3 — Quality control and trimming

Goal: Assess raw read quality, trim adapters/low-quality bases, confirm improvement.

Tools: FastQC, MultiQC, fastp.

Commands

```
# scripts/02_qc_trim.sh
mkdir -p results/qc/raw results/qc/trimmed data/trimmed

# 1) Raw QC
fastqc data/raw/*.fastq.gz -o results/qc/raw -t 4
multiqc results/qc/raw -o results/qc/raw

# 2) Trim each pair with fastp (adapter + quality + auto report)
for R1 in data/raw/*_1.fastq.gz; do
  SAMP=$(basename "$R1" _1.fastq.gz)
  fastp \
    -i "data/raw/${SAMP}_1.fastq.gz" -I "data/raw/${SAMP}_2.fastq.gz" \
    -o "data/trimmed/${SAMP}_1.trim.fastq.gz" -O "data/trimmed/${SAMP}_2.trim.fastq.gz" \
    --detect_adapter_for_pe --qualified_quality_phred 20 --length_required 25 \
    --thread 4 --html "results/qc/trimmed/${SAMP}_fastp.html" \
    --json "results/qc/trimmed/${SAMP}_fastp.json"
done

# 3) Post-trim QC summary
fastqc data/trimmed/*.trim.fastq.gz -o results/qc/trimmed -t 4
multiqc results/qc/trimmed -o results/qc/trimmed
```

Expected output: `results/qc/raw/multiqc_report.html` and `results/qc/trimmed/multiqc_report.html`. After trimming you should see adapter content drop to near-zero and per-base quality improve. Keep both reports — the before/after comparison is a great README figure.

Day 4 — Alignment to GRCh38

Goal: Map trimmed reads to the genome and produce sorted, indexed BAMs.

Tools: HISAT2, samtools. 🚩 >4h >4h likely across 12 samples.

HISAT2 on human needs ~6–8 GB RAM — fine for 16 GB, but each sample takes roughly 15–40 min, so 12 samples ≈ 3–8 h. Mitigations: run in the background/overnight; process 4+4 samples; or use the lighter Salmon alternative (quasi-mapping, ~10× faster, no big RAM) shown at the end of this day. Keep HISAT2 as the primary path because the job description explicitly says "alignment."

```
# scripts/03_align_count.sh (alignment portion)
mkdir -p data/bam
IDX=data/reference/grch38/genome # adjust if the tarball extracts a different prefix

for R1 in data/trimmed/*_1.trim.fastq.gz; do
  SAMP=$(basename "$R1" _1.trim.fastq.gz)
  hisat2 -p 4 -x "$IDX" \
    -1 "data/trimmed/${SAMP}_1.trim.fastq.gz" \
    -2 "data/trimmed/${SAMP}_2.trim.fastq.gz" \
    --new-summary --summary-file "results/qc/${SAMP}.hisat2.log" \
    | samtools sort -@ 4 -o "data/bam/${SAMP}.sorted.bam" -
  samtools index "data/bam/${SAMP}.sorted.bam"
done

multiqc results/qc -o results/qc # aggregates HISAT2 alignment rates
```

Lighter alternative — Salmon (use if **alignment is too slow**; note it in your README as a comparison):

```
conda install -n rnaseq -c bioconda salmon
# build transcriptome index once from GENCODE transcripts, then:
# salmon quant -i salmon_index -l A -1 R1 -2 R2 -p 4 --validateMappings -o quant/${SAMP}
```

Expected output: `data/bam/*.sorted.bam` + `.bai` for each sample; **HISAT2 logs** showing overall alignment rate (expect ~85–95% for good human RNA-seq). The **MultiQC report now includes alignment stats**.

Day 5 — Gene-level counting and count-matrix assembly

Goal: Turn **BAMs** into a **gene × sample count matrix**; validate against published counts.

Tools: featureCounts (subread), R/pandas for matrix assembly.

```
# scripts/03_align_count.sh (counting portion)
mkdir -p results/tables
featureCounts -T 4 -p --countReadPairs -s 0 \
  -a data/reference/genome.v44.annotation.gtf \
  -o results/tables/subset_counts.txt \
  data/bam/*.sorted.bam
```

Notes: `-p --countReadPairs` for paired-end; `-s 0` = **unstranded** (check your library's strandedness in the fastp/MultiQC output — if reverse-stranded use `-s 2`). The output's first columns are **gene metadata**; **sample counts start at column 7**.

Validate the pipeline (the key Track-A ↔ Track-B check): load `subset_counts.txt`, strip to **gene × sample**, and **correlate per-gene log-counts** against the same samples in `GSE164416_counts.txt`. A Spearman/Pearson $r > \sim 0.95$ confirms your pipeline reproduces the published processing. **Put that scatter plot in the README.**

Expected output: `results/tables/subset_counts.txt` (your own matrix) and a **validation correlation $\geq \sim 0.95$** vs the published matrix.

Day 6 — Differential expression with DESeq2

Goal: ND vs T2D differential expression on the **full** published matrix (Track B).

Tools: R, DESeq2, EnhancedVolcano.

Build a metadata table (`data/processed/metadata.csv`) with two columns, `sample,condition` , `parsing` ND/T2D from the count-matrix column names. Then:

```
# scripts/04_deseq2.R (run in the rstats env: conda activate rstats)
library(DESeq2); library(EnhancedVolcano); library(data.table)

cts <- as.matrix(fread("data/processed/GSE164416_counts.txt"), rownames = 1)
meta <- read.csv("data/processed/metadata.csv", row.names = 1)
meta <- meta[meta$condition %in% c("ND","T2D"), , drop = FALSE] # focus contrast
cts <- cts[, rownames(meta)]
meta$condition <- factor(meta$condition, levels = c("ND","T2D"))

dds <- DESeqDataSetFromMatrix(cts, meta, design = ~ condition)
dds <- dds[rowSums(counts(dds)) >= 10, ] # low-count filter
dds <- DESeq(dds)
res <- results(dds, contrast = c("condition","T2D","ND"))
res <- res[order(res$padj), ]
write.csv(as.data.frame(res), "results/tables/deseq2_T2D_vs_ND.csv")

# PCA sanity check
vsd <- vst(dds, blind = TRUE)
pdf("results/figures/pca.pdf"); plotPCA(vsd, intgroup = "condition"); dev.off()

# Volcano
pdf("results/figures/volcano.pdf", width = 7, height = 6)
EnhancedVolcano(res, lab = rownames(res), x = "log2FoldChange", y = "padj",
  pCutoff = 0.05, FCcutoff = 1.0, title = "T2D vs ND")
dev.off()
```

Expected output: `deseq2_T2D_vs_ND.csv` (full results), `volcano.pdf` , `pca.pdf` . PCA should show ND and T2D at least partially separating; the volcano highlights significant up/down genes (`padj < 0.05`, `|log2FC| > 1`). Gene IDs will be `Ensembl/GENCODE IDs` — you'll map them to symbols next.

Day 7 — Split mRNA vs lncRNA, filter DE sets, heatmap

Goal: Classify each DE gene by `biotype`, extract **DE mRNAs** and **DE lncRNAs**, visualize.

Tools: R, biomaRt (or the GENCODE GTF), pheatmap.

The GENCODE GTF carries a `gene_type` attribute per gene (`protein_coding` , `lncRNA` , `miRNA` , etc.). Parse it into a lookup, or use biomaRt:

```
# scripts/05_biotype_split.R
library(biomaRt); library(dplyr); library(pheatmap)
res <- read.csv("results/tables/deseq2_T2D_vs_ND.csv", row.names = 1)
res$ensembl <- sub("\\..*$", "", rownames(res)) # strip version suffix

mart <- useEnsembl("genes", dataset = "hsapiens_gene_ensembl")
ann <- getBM(attributes = c("ensembl_gene_id", "external_gene_name", "gene_biotype"),
             filters = "ensembl_gene_id", values = res$ensembl, mart = mart)
res <- left_join(res, ann, by = c("ensembl" = "ensembl_gene_id"))

sig <- subset(res, padj < 0.05 & abs(log2FoldChange) > 1)
de_mrna <- subset(sig, gene_biotype == "protein_coding")
de_lncrna <- subset(sig, gene_biotype == "lncRNA")
write.csv(de_mrna, "results/tables/DE_mRNA.csv", row.names = FALSE)
write.csv(de_lncrna, "results/tables/DE_lncRNA.csv", row.names = FALSE)

# Heatmap of top 50 DE genes (needs the vst matrix from Day 6 – save it there)
# top <- head(order(sig$padj), 50); pheatmap(vst_mat[top,], ... ) -> results/figures/
heatmap.pdf
```

Expected output: `DE_mRNA.csv` and `DE_lncRNA.csv` with gene symbols and biotypes; a clustered `heatmap.pdf` of top DE genes showing ND vs T2D separation. **Report counts** (e.g. "N DE mRNAs, M DE lncRNAs").

Day 8 — Functional enrichment (biological story)

Goal: GO and **KEGG** enrichment on DE mRNAs — gives your project a **narrative**, which **interviewers love**.

Tools: R, clusterProfiler, org.Hs.eg.db.

```
# scripts/06_enrichment.R
library(clusterProfiler); library(org.Hs.eg.db); library(ggplot2)
de <- read.csv("results/tables/DE_mRNA.csv")
eg <- bitr(de$external_gene_name, "SYMBOL", "ENTREZID", org.Hs.eg.db)

ego <- enrichGO(eg$ENTREZID, org.Hs.eg.db, ont = "BP",
                pAdjustMethod = "BH", qvalueCutoff = 0.05, readable = TRUE)
kk <- enrichKEGG(eg$ENTREZID, organism = "hsa", qvalueCutoff = 0.05)

write.csv(as.data.frame(ego), "results/tables/GO_BP.csv")
write.csv(as.data.frame(kk), "results/tables/KEGG.csv")
ggsave("results/figures/GO_dotplot.pdf", dotplot(ego, showCategory = 20), width = 8,
        height = 8)
```

Expected output: `GO_BP.csv`, `KEGG.csv`, `GO_dotplot.pdf`. For **islet T2D data** expect terms around **insulin** secretion, hormone/peptide processing, ion transport, and metabolism.

Day 9 — miRNA layer: predict the connecting miRNAs

Goal: Find the miRNAs that (a) **target your DE mRNAs** and (b) **interact with your DE lncRNAs**. This is the **bridge of the ceRNA network**.

Tools: R **multiMiR** (queries miRTarBase/TargetScan/miRDB for **miRNA → mRNA**); ENCORI/starBase for lncRNA ↔ miRNA.

```
# scripts/07_cerna_predict.R
library(multiMiR); library(dplyr)

# miRNA -> mRNA: validated + predicted interactions for your DE mRNAs
mrna <- read.csv("results/tables/DE_mRNA.csv")
mm <- get_multimir(org = "hsa", target = mrna$external_gene_name,
                  table = "validated", summary = TRUE)
mir_mrna <- mm@data %>% select(mature_mirna_id, target_symbol) %>% distinct()
write.csv(mir_mrna, "results/tables/miRNA_mRNA_edges.csv", row.names = FALSE)
```

For **lncRNA → miRNA**, use ENCORI/starBase (<https://rnasysu.com/encori/>): download the **"lncRNA–miRNA" interaction table for your DE lncRNAs**, or **use the miRcode predicted-targets table** (<http://www.mircode.org/>) which is a single downloadable file you can **filter offline**. Save as **results/tables/lncRNA_miRNA_edges.csv** with columns **lncRNA, mature_mirna_id**.

Expected output: **miRNA_mRNA_edges.csv** (**miRNA → DE-mRNA**) and **lncRNA_miRNA_edges.csv** (**DE-lncRNA → miRNA**). The **shared miRNAs** between these two tables are the hinges of the ceRNA network.

Day 10 — Build the ceRNA network

Goal: Assemble the **lncRNA–miRNA–mRNA network** by connecting the **two** edge tables through shared miRNAs.

Tools: R, dplyr, igraph.

```
# scripts/08_network.R (assembly portion)
library(dplyr); library(igraph)
lm <- read.csv("results/tables/lncRNA_miRNA_edges.csv") # lncRNA -- miRNA
mm <- read.csv("results/tables/miRNA_mRNA_edges.csv")   # miRNA -- mRNA

shared <- intersect(lm$mature_mirna_id, mm$mature_mirna_id)
lm <- lm %>% filter(mature_mirna_id %in% shared)
mm <- mm %>% filter(mature_mirna_id %in% shared)

edges <- bind_rows(
  lm %>% transmute(from = lncRNA, to = mature_mirna_id, type = "lncRNA-miRNA"),
  mm %>% transmute(from = mature_mirna_id, to = target_symbol, type = "miRNA-mRNA")
)
write.csv(edges, "results/tables/ceRNA_edges.csv", row.names = FALSE)

g <- graph_from_data_frame(edges, directed = FALSE)
V(g)$degree <- degree(g)
write.csv(data.frame(node = V(g)$name, degree = V(g)$degree),
          "results/tables/ceRNA_node_degree.csv", row.names = FALSE)
```

Expected output: **ceRNA_edges.csv** (the full lncRNA–miRNA–mRNA edge list) and **ceRNA_node_degree.csv**. Only **triads** sharing a **real miRNA survive**, so this is a **genuine competing-endogenous-RNA network** grounded in your DE results.

Day 11 — Network visualization and hub analysis

Goal: A publication-style network figure and identification of hub nodes.

Tools: igraph (quick, scriptable) and/or Cytoscape (prettier, manual). 🚩 >4h Cytoscape layout tuning can eat time — budget it, don't perfect it.

```
# scripts/08_network.R (plot portion)
node_type <- function(n) ifelse(grepl("^hsa-", n), "miRNA",
                                ifelse(n %in% read.csv("results/tables/DE_lncRNA.csv")$external_ge
ne_name,
                                "lncRNA", "mRNA"))
V(g)$type <- sapply(V(g)$name, node_type)
V(g)$color <- c(lncRNA = "tomato", miRNA = "gold", mRNA = "skyblue")[V(g)$type]
pdf("results/figures/cerna_network.pdf", width = 10, height = 10)
plot(g, vertex.size = 3 + V(g)$degree, vertex.label.cex = 0.5,
     layout = layout_with_fr(g), main = "T2D vs ND ceRNA network")
dev.off()
```

For Cytoscape: **File → Import → Network from File → cerna_edges.csv**, import **cerna_node_degree.csv** as a node table, size/color by degree and type, apply a **force-directed layout**, export PNG. Report the top hub miRNAs/lncRNAs by degree — these are your **candidate regulators**.

Expected output: **cerna_network.pdf** (or Cytoscape PNG) with **three colored node classes**, and a short list of hub nodes.

Day 12 — Reproducibility and script cleanup

Goal: Make everything **re-runnable by a stranger**.

Tools: conda, bash, (optional) **Snakemake** or a single **run_all.sh**.

- **Clean** each **scripts/0*.sh / .R** so **paths are relative** and **parameters are at the top**.
- Add a **top-level run_all.sh** **documenting the exact order**.
- **Re-export both envs:** **conda env export -n rnaseq --no-builds > environment.yml** and **record R packages** in **docs/methods.md**.
- Optional stretch: wrap the **FASTQ → counts steps in a small Snakefile** — **a big signal of pipeline maturity**.

Expected output: A clean **scripts/** folder, **run_all.sh**, and an updated **environment.yml** committed to Git.

Day 13 — README and documentation

Goal: The README that a hiring PI will actually read.

Structure it as: **one-line summary** → **dataset + accession** → **pipeline diagram** (**QC → trim → align → count → DE → biotype → ceRNA**) → **how to reproduce** (**conda env create -f environment.yml**, then **run_all.sh**) → **key results with embedded figures** (volcano, heatmap, GO dotplot, network) → the **Track-A vs Track-B validation scatter** → **limitations** (miRNA layer is predicted, small subset aligned locally) → **references**. Write **docs/methods.md** with **tool versions and exact parameters**.

Expected output: A polished `README.md` with images rendering on GitHub, and `docs/methods.md`.

Day 14 — Interpretation, sanity checks, and a short report

Goal: Verify results and write the biology up.

- Sanity-check: are known **T2D/islet genes** (e.g. **INS**, and **stress/dedifferentiation markers**) moving in sensible directions? Cross-check a few DE hits against the literature.
- Confirm the **ceRNA hubs** make biological sense (**search 2–3 hub miRNAs for known diabetes roles**).
- Write a 1–2 page `docs/report.md` (or PDF) **summarizing question → methods → key findings → figures → limitations**.

Expected output: A short written report and a documented list of validated sanity checks.

Day 15 — Final polish, push, and CV framing

Goal: Ship it and connect it to the application.

- Final `git add -A && git commit && git push`; check every figure renders on GitHub and `environment.yml` installs cleanly in a fresh env.
- Add **repo topics/description**; **pin it on your GitHub profile**.
- Draft 3–4 CV bullets and a short paragraph for your statement of research interests that map directly onto the job's language: **"identification and characterization of miRNAs and lncRNAs, prediction of gene targets, construction of ceRNA networks."**
- In your **cover email** to Dr. Mohsin, **link the repo and note you can extend the pipeline to matched small-RNA-seq for experimentally-anchored miRNAs and to qRT-PCR validation of hub axes — showing you understand the wet-lab follow-up in the role**.

Expected output: A public, reproducible GitHub repo and application-ready framing tying the project to the RA position.

Time-risk summary (the >4h flags)

Step	Day	Risk	Mitigation
FASTQ download (12 samples, 20–40 GB)	2	🚩 >4h >4h on slow internet	Run overnight; use ENA; drop to 4+4
HISAT2 alignment (12 samples)	4	🚩 >4h >4h likely	Background/overnight; fewer samples; or Salmon
Genome index build	4	🚩 >4h >4h if self-built	Use the prebuilt index (as instructed)
Cytoscape layout tuning	11	Time sink	Time-box it; igraph plot is enough

Everything else (QC, trimming, counting, DESeq2, biotype split, enrichment, prediction, network assembly) runs in **minutes** on 16 GB.

Skills-to-job-description map

Job requirement	Where you demonstrate it
RNA-seq analysis, diabetic vs control	Days 2–6 (full pipeline + DESeq2)
miRNA & lncRNA identification	Days 7, 9
Gene-target prediction	Day 9 (multiMiR / TargetScan / miRTarBase)
ceRNA network construction	Days 10–11
Reproducible bioinformatics	Days 12–13 (conda, scripts, README)
qRT-PCR / validation awareness	Day 15 framing (offer as next step)

Key references

- GSE164416 — <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE164416>
- Wigger et al., *Nat Metab* 2021 (PMID 34183850)
- multiMiR — <https://bioconductor.org/packages/multiMiR/>
- ENCORI/starBase — <https://rnasysu.com/encori/>
- miRcode — <http://www.mircode.org/>
- GENCODE — <https://www.gencodegenes.org/>